

Conceptual Session 2

JavaScript Fundamentals with a Problem-Solving Mindset

Concept Notes & Practice Problem Statements

Type A: solved live by the instructor | Type B: solved by students

This document contains problem statements only — no solutions.

Working with Text & Objects

Topics Covered (Concept Explanation)

Strings - Text as Data

A string is simply text — a sequence of characters wrapped in single quotes, double quotes, or backticks. Strings share some behavior with arrays: both have a `.length` property, and both allow you to access an individual item (a character, for a string) using square-bracket index notation starting at 0. The key difference is mutability — array elements can be directly reassigned by index (`arr[0] = ...`), but a string's individual characters cannot be changed this way; any 'modification' of a string actually creates a brand-new string rather than editing the original in place.

```
let word = "Hero";
console.log(word.length); // 4
console.log(word[0]);     // "H"
// word[0] = "Z"; // does NOT change the string
```

Cleaning & Comparing Text

Because JavaScript string comparisons are case-sensitive by default (`"Dhaka" !== "dhaka"`), `toLowerCase()` and `toUpperCase()` are used to normalize two strings to the same case before comparing them — a very common real-world need, such as checking a username or email regardless of how the user typed it. `trim()` removes any whitespace from the very start and end of a string (but not spaces in the middle), which is essential for cleaning up user input that may have accidental leading/trailing spaces before comparing or storing it.

```
let a = " Dhaka ";
let b = "dhaka";
console.log(a.trim().toLowerCase() === b); // true
```

Extracting & Joining Text

`slice(start, end)` extracts a portion of a string into a new string, counting from the start index up to (but not including) the end index — omitting the end index slices to the end of the string. Strings can be joined together (concatenation) using the `+` operator, the `.concat()` method, or — most commonly in modern JavaScript — template literals, which use backticks and `${ }` to embed variables directly inside a string. `includes()` checks whether one string contains another substring anywhere inside it, returning `true` or `false`, which is extremely useful for search-style problems.

```
let name = "Hero";
console.log(name.slice(0, 2)); // "He"
let full = "Programming " + name; // concatenation with +
let msg = `Welcome, ${name}!`;    // template literal
console.log(name.includes("ero")); // true
```

Flipping a String

Since strings don't have a built-in `.reverse()` method the way arrays do, reversing one requires a bit more thought — and there are three common approaches students should recognize. Method 1: convert the string to an array of characters with `split("")`,

reverse that array with `.reverse()`, and rejoin it into a string with `.join("")`. Method 2: loop through the string from the last character to the first, manually building up a new string one character at a time. Method 3 (more advanced): use recursion, where the function reverses a smaller and smaller piece of the string on each call until it reaches the base case. All three reach the same result, but they build different problem-solving muscles.

```
let str = "Hero";
let reversed = str.split("").reverse().join("");
console.log(reversed); // "oreH"
```

Javascript Object

An object is a collection of related data stored as key-value pairs, called properties — unlike an array, which is an ordered list accessed by numeric index, an object is accessed by named keys, making it ideal for describing a single 'thing' with multiple attributes (like a student having a name, age, and grade all in one place). The animated video reinforces this visually, showing an object as a labeled folder containing several labeled compartments, each holding one piece of related information.

```
let student = {
  name: "Ayan",
  age: 20,
  isPassed: true,
};
```

Exploring & Editing Objects

An object's property values can be read or updated in two ways: dot notation (`object.propertyName`), which is shorter and used most of the time, and bracket notation (`object["propertyName"]`), which is required when the property name is stored in a variable, contains spaces, or is being generated dynamically. Both notations work identically for simple, fixed property names, so choosing between them usually comes down to whether the property name is known ahead of time or needs to be computed.

```
let student = { name: "Ayan", age: 20 };
console.log(student.name);           // dot notation -> "Ayan"
console.log(student["age"]);         // bracket notation -> 20
student.age = 21;                    // update via dot notation
let key = "name";
console.log(student[key]);            // bracket notation with a variable
```

Keys, Values, Nested Objects & delete

`Object.keys(obj)` returns an array of all of an object's property names, and `Object.values(obj)` returns an array of all its corresponding values — both are useful when you need to inspect or loop through an object generically. Objects can also be nested, meaning one object's property value is itself another object (e.g., a student object containing an address object) — nested properties are accessed by chaining dot or bracket notation (`student.address.city`). The delete operator permanently removes a specified property from an object.

```
let student = {
  name: "Ayan",
  address: { city: "Dhaka", zip: "1207" },
};
console.log(Object.keys(student));    // ["name", "address"]
console.log(student.address.city);    // "Dhaka"
delete student.address.zip;           // removes the zip property
```

Looping an Object & Different Ways to Declare an Object

Because objects aren't ordered lists, a regular for loop (which relies on numeric indexes) can't traverse them — instead, the `for...in` loop is used, which goes through each key in the object one at a time, letting you look up its matching value inside the loop body. Objects can be declared in a couple of ways: the common object literal syntax (`{ key: value }`) used throughout this module, or `new Object()` combined with assigning properties afterward — the literal syntax is simpler and used far more often in practice.

```
let student = { name: "Ayan", age: 20 };
for (let key in student) {
  console.log(key, ":", student[key]);
}
```

Type A — Instructor Will Solve Live in Session

1. Given a string, print its length and the character found at a specific index.
2. Take a string with extra leading/trailing spaces and mixed casing, and use `trim()` and `toLowerCase()` to clean it up, then print the cleaned result.
3. Given two strings (first name and last name), combine them into a full name using a template literal and print a greeting message.
4. Check whether a given string includes a specific word or substring, and print true or false.
5. Reverse a given string using the `split() → reverse() → join()` method.
6. Create an object representing a student with name, age, and grade properties, then print each property using dot notation.
7. Given an object representing a book, use `Object.keys()` and `Object.values()` to print all of its property names and values separately.
8. Given a nested object representing a person with an embedded address object, access and print a property from inside the nested object.

Type B — Students Will Solve

1. Given a string, print it in both uppercase and lowercase using the appropriate string methods.
2. Given a string with leading and trailing spaces, print its length before trimming and after trimming to show the difference.
3. Use `slice()` to extract and print the first three characters and the last three characters of a given string.
4. Reverse a given string manually using a for loop (without using `split/reverse/join`).
5. Create an object representing a product with name, price, and quantity, then update the price using bracket notation and print the updated object.
6. Given an object, use the delete operator to remove one of its properties and print the object before and after deletion.
7. Write a program that loops through an object representing a person's details using a `for...in` loop and prints each key alongside its value.
8. Given a sentence (string), count and print how many times a specific character appears in it.

Fundamentals of Functions

Topics Covered (Concept Explanation)

Javascript Functions

A function is a reusable, named block of code that performs a specific task — instead of rewriting the same lines every time we need that task done, we define the function once and call (invoke) it as many times as needed. The basic syntax pairs the function keyword with a name and parentheses (which may hold parameters) followed by a curly-brace block containing the code to run. The animated video reinforces functions as a kind of 'machine': you feed something in, the machine does its defined job, and (optionally) something comes back out.

```
function greet() {  
  console.log("Hello, welcome!");  
}  
greet();    // calling/invoking the function  
greet();    // can be called again anytime
```

Function Parameters

Parameters are named placeholders listed inside a function's parentheses that let the function accept input rather than always doing exactly the same fixed thing. A function can define multiple parameters separated by commas, and each one becomes a local variable available for use inside that function's body — this is what allows a single function like `calculateTotal(price, quantity)` to work correctly for any price and quantity it's given.

```
function addNumbers(a, b) {  
  console.log(a + b);  
}  
addNumbers(5, 10);    // 15  
addNumbers(2, 3);    // 5
```

Parameters vs Arguments

Parameters and arguments are closely related but distinct: parameters are the placeholder names written in the function's own definition, while arguments are the actual values supplied when the function is called. In `addNumbers(a, b)`, `a` and `b` are parameters; in the call `addNumbers(5, 10)`, the numbers `5` and `10` are arguments. When a function is called, JavaScript matches each argument to its corresponding parameter in order, runs the function's code with those values plugged in, and then returns control back to wherever the function was called from.

Getting a Result Back

The `return` keyword sends a value back out of a function to whatever code called it, and it also immediately ends the function's execution at that point (any code after a `return` statement inside the same block never runs). A function's returned value can be stored in a variable, used directly in a calculation, or printed — and importantly, a function with no explicit `return` statement automatically returns `undefined`, which is a common source of beginner bugs when a function is expected to produce a usable value.

```
function square(n) {  
  return n * n;  
}  
let result = square(5);  
console.log(result);    // 25
```

Decisions Inside Functions

This section combines everything so far — parameters, arguments, and `return` — with the conditional logic from Module 8, using a classic example: a function that accepts a number and returns the string "Even" or "Odd" depending on the result of a

modulus check. This is a useful checkpoint for confirming students can comfortably combine functions with decision-making logic rather than treating them as two separate topics.

```
function checkEvenOdd(num) {
  if (num % 2 === 0) {
    return "Even";
  } else {
    return "Odd";
  }
}
console.log(checkEvenOdd(7)); // "Odd"
```

Optional Inputs with Defaults

Beyond plain required parameters, JavaScript supports default parameters, where a parameter is given a fallback value that's used automatically if no argument (or undefined) is passed in for it — useful for optional settings like a tax rate or discount that usually stays the same. Functions can also mix required and default parameters together, and can accept parameters of different data types (numbers, strings, Booleans, even arrays or objects) depending on what the task needs.

```
function calculatePrice(price, taxRate = 0.05) {
  return price + price * taxRate;
}
console.log(calculatePrice(100)); // uses default 0.05 -> 105
console.log(calculatePrice(100, 0.1)); // uses given 0.1 -> 110
```

Functions That Process a List

This topic combines functions with the array traversal skills from Module 10 — writing a function that accepts an array as a parameter, loops through every element to accumulate a running total, and returns that total once the loop finishes. It's a foundational pattern (accept a collection → process every item → return one combined result) that reappears constantly in real-world programming.

```
function sumArray(arr) {
  let total = 0;
  for (let i = 0; i < arr.length; i++) {
    total += arr[i];
  }
  return total;
}
console.log(sumArray([1, 2, 3, 4])); // 10
```

Extending the previous pattern, this topic writes a function that accepts an array, loops through it, uses a conditional to test each element, and — instead of accumulating a single total — builds and returns a brand-new array containing only the elements that passed the test (in this case, even numbers). This 'loop + condition + collect matching results into a new array' pattern is one of the most reusable problem-solving templates students will carry forward.

```
function getEvenNumbers(arr) {
  let evens = [];
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] % 2 === 0) {
      evens.push(arr[i]);
    }
  }
  return evens;
}
console.log(getEvenNumbers([1,2,3,4,5,6])); // [2, 4, 6]
```

1. Write a function that takes a name as a parameter and returns a greeting message, then print the returned message.
2. Write a function that takes two numbers as parameters and returns their sum.
3. Write a function with a default parameter for tax rate (5%) that calculates and returns the total price of an item including tax.
4. Write a function that takes a number as a parameter and returns whether it is 'Even' or 'Odd'.
5. Write a function that takes an array of numbers as a parameter and returns the sum of all the elements.
6. Write a function that takes three numbers as parameters and returns the largest of the three.
7. Write a function that takes a string as a parameter and returns the number of characters (its length).
8. Write a function that takes an array of numbers as a parameter and returns the average of all the elements.

Type B — Students Will Solve

1. Write a function that takes a person's name and age as parameters and returns a formatted introduction sentence.
2. Write a function that takes the length and width of a rectangle as parameters and returns its area.
3. Write a function with a default parameter for discount percentage (10%) that returns the discounted price of an item.
4. Write a function that takes an array of numbers as a parameter and returns a new array containing only the even numbers.
5. Write a function that takes a number as a parameter and returns its factorial.
6. Write a function that takes a string as a parameter and returns the number of vowels found in it.
7. Write a function that takes an array of numbers as a parameter and returns a new array containing only the numbers greater than 10.
8. Write a function that takes two numbers as parameters and returns the larger of the two without using `Math.max()`.